



CAI-RUNTS — Release 001

Documento di specifica di rilascio

*Release 001 — v1.1
23 maggio 2026*

*Stato: **draft***

Sommario

Primo rilascio incrementale dopo la fotografia v1.0. Ruota intorno a quattro macro-temi: **mappa potenziata** (vista d'insieme su tutto il dataset filtrato, clustering, filtri live), **export del dato** (CSV/Excel della lista, PDF della scheda ente con carta intestata Montagna Servizi), **resilienza dello scraper** (retry su errori transitori, preservazione delle coordinate già geocodificate) e **performance e cache** (indici DB, cache locale del geocoding).

Nessuna modifica disruptive dello schema: tutte le evoluzioni del DB sono additive e idempotenti tramite la consueta lista `_MIGRATIONS`.

Motivazione (Why)

- La **mappa attuale** mostra al massimo 20 marker (la pagina corrente) e per i ~226 enti CAI rende la vista d'insieme inefficace. Senza clustering, una mappa con centinaia di marker diventerebbe comunque illeggibile in città con più sezioni.
- Gli operatori CAI e Montagna Servizi hanno bisogno di **esportare** dataset e schede per analisi offline, presentazioni, condivisione con referenti regionali. Oggi l'unico canale è il browser.
- Il **rerun dello scraper azzera** `lat / lon` di tutti gli enti aggiornati (problema noto, vedi `03-database.md`): obbliga a rilanciare il geocoder a ogni run con un costo di ~4 minuti su Nominatim, e per il tempo intermedio la mappa è parziale.
- Un singolo timeout di rete oggi salta un ente dal batch; con un retry semplice si recupera la maggior parte di questi casi.
- Con il dataset che probabilmente crescerà (oltre CAI: altre reti del Terzo Settore), gli **indici** sui filtri principali diventano un prerequisito di scalabilità.

Cosa cambia (What)

Utente finale

- **Mappa "tutti gli enti"**: nella vista mappa della lista compaiono i marker di tutti gli enti che soddisfano i filtri correnti, non solo quelli della pagina visibile.
- **Cluster**: marker raggruppati con conteggio quando vicini; al click si espandono fino a mostrare i singoli enti.
- **Sidebar filtri sulla mappa**: pannello laterale con elenco regioni e sezioni del registro, con conteggio per opzione; selezionarle filtra i marker in tempo reale senza ricaricare la pagina e sincronizza l'URL.
- **Pulsanti di export** in pagina lista: "Esporta CSV" e "Esporta Excel" che producono il dataset filtrato.

- **Pulsante "Scarica scheda PDF"** in pagina dettaglio ente: produce un PDF con carta intestata Montagna Servizi che riporta tutti i campi non nulli dell'ente più la mappa della sede.

Backend / dati

- **Nuovo endpoint** `GET /api/enti.geojson?<filtri>` : restituisce GeoJSON `FeatureCollection` con tutti gli enti del filtro corrente che hanno coordinate valorizzate.
- **Nuovo endpoint** `GET /api/enti.csv?<filtri>` e `GET /api/enti.xlsx?<filtri>` per gli export tabellari.
- **Nuovo endpoint** `GET /ente/{id_runts}/pdf` per la scheda PDF.
- `upsert_ente` **preserva** `lat / lon` se nel dict in arrivo sono `None` o assenti.
- **Scraper retry**: in caso di timeout o errore di rete su un ente, ritenta fino a 3 volte con backoff esponenziale (1 s, 2 s, 4 s).
- **Nuova tabella** `geocoding_cache` in `runts.db` con chiave normalizzata `comune|provincia|regione` → `(lat, lon, source, ts)` . Il geocoder cerca prima nella cache, ricade su Nominatim solo in caso di miss.
- **Indici DB** su `enti(sede_region)` e `enti(sezione_registro)` .

Requisiti funzionali

A. Mappa potenziata

A1. Il sistema SHALL esporre l'endpoint `GET /api/enti.geojson` che restituisce un GeoJSON `FeatureCollection` contenente **tutti** gli enti del DB che corrispondono ai filtri della query string (`q` , `regione` , `sezione_registro`) e che hanno `lat` e `lon` valorizzati, **senza paginazione**.

A2. Ogni `Feature` SHALL contenere geometria `Point ( [lon, lat] )` e proprietà `id_runts` , `denominazione` , `sede_comune` , `sede_region` , `sezione_registro` .

A3. La vista mappa della pagina `/` SHALL caricare i marker tramite chiamata fetch al nuovo endpoint geojson invece di leggerli dall'array inline.

A4. I marker SHALL essere visualizzati tramite `Leaflet.markercluster` con cluster automatico per zoom. Il cluster mostra il conteggio degli enti contenuti.

A5. Il sistema SHALL mostrare nella vista mappa una **sidebar filtri** con due liste a checkbox: regioni e sezioni del registro, ciascuna con il conteggio di enti corrispondenti.

A6. Al cambio di una checkbox della sidebar, il sistema SHALL filtrare i marker visibili in **tempo reale lato client** (senza ricaricare la pagina) e aggiornare la query string del browser tramite `history.replaceState` .

B. Export del dato

B1. Il sistema SHALL esporre l'endpoint `GET /api/enti.csv` che restituisce un file CSV (UTF-8 con BOM, separatore `;`) con i campi: `id_runts`, `denominazione`, `codice_fiscale`, `sede_indirizzo`, `sede_civico`, `sede_comune`, `sede_provincia`, `sede_region`, `sede_cap`, `sezione_registro`, `forma_giuridica`, `natura_giuridica`, `data_iscrizione`, `pec`, `sito_web`, `url_dettaglio`, `lat`, `lon`.

B2. Il sistema SHALL esporre l'endpoint `GET /api/enti.xlsx` che restituisce lo stesso dataset di B1 in formato Excel (un foglio "Enti", header congelato).

B3. Gli endpoint di B1/B2 SHALL accettare gli stessi parametri di filtro della lista (`q`, `regione`, `sezione_registro`) e applicarli identicamente.

B4. Il sistema SHALL esporre l'endpoint `GET /ente/{id_runts}/pdf` che restituisce un PDF della scheda dell'ente con i seguenti elementi: copertina con denominazione e codice fiscale, tabella di tutti i campi non nulli, mappa della sede legale (se `lat` / `lon` presenti), tutto sovrapposto alla carta intestata Montagna Servizi.

B5. I template `list.html` e `detail.html` SHALL esporre, rispettivamente, i pulsanti "Esporta CSV", "Esporta Excel" e "Scarica scheda PDF".

C. Resilienza scraper e persistenza coordinate

C1. La funzione `upsert_ente` SHALL preservare i valori esistenti di `lat` e `lon` quando il dict in input non li contiene oppure li contiene a `None`. In altre parole: l' `INSERT` OR `REPLACE` SHALL leggere i valori correnti prima di sovrascrivere e applicare un **MERGE** sui soli campi `lat` / `lon`.

C2. Lo scraper SHALL, in caso di eccezione durante l'estrazione del dettaglio di un singolo ente, **ritentare fino a 3 volte** la stessa estrazione con backoff esponenziale (1 s, 2 s, 4 s).

C3. Se dopo i 3 tentativi l'estrazione fallisce ancora, lo scraper SHALL loggare ERROR con `id_runts` e denominazione, e proseguire con l'ente successivo (comportamento attuale).

C4. Il report finale dello scraper SHALL riportare il conteggio degli enti recuperati al primo tentativo, al secondo, al terzo e definitivamente falliti.

D. Performance e cache geocoding

D1. Il sistema SHALL creare gli indici `idx_enti_sede_region` su `enti(sede_region)` e `idx_enti_sezione_registro` su `enti(sezione_registro)` tramite migrazione idempotente.

D2. Il sistema SHALL creare la tabella `geocoding_cache(cache_key TEXT PRIMARY KEY, lat REAL NOT NULL, lon REAL NOT NULL, source TEXT NOT NULL, ts TEXT NOT NULL)`, dove `cache_key` è la concatenazione normalizzata in lowercase `<comune>|<provincia>|<regione>`.

D3. Il geocoder SHALL, prima di chiamare Nominatim, eseguire una `SELECT` sulla `geocoding_cache` con la `cache_key` del record corrente; se la cache hit ritorna coordinate, le applica senza chiamata HTTP.

MONTAGNA SERVIZI S.C.P.A.

Via Errico Petrella 19 - 20124 - Milano (MI)

<https://montagnaservizi.com>

PEC: montagnaserviziscpa@legalmail.it - email: info@montagnaservizi.com

C.F./P.IVA: 11790660960

D4. Il geocoder SHALL, in caso di miss in cache e successo Nominatim, inserire la riga nella `geocoding_cache` con `source='nominatim'` e `ts` ISO UTC.

D5. Il report finale del geocoder SHALL distinguere "da cache" e "da Nominatim" nel conteggio dei geocodificati.

Requisiti non funzionali

- **Performance:** `GET /api/enti.geojson` con i 226 enti attuali SHALL rispondere in < 200 ms su hardware di sviluppo standard. Con gli indici di D1, una query filtrata su regione+sezione SHALL essere $O(\log n)$.
- **Compatibilità DB:** tutte le modifiche allo schema SHALL essere idempotenti tramite `_MIGRATIONS`, senza necessità di reset del file `runts.db`.
- **Read-only lato web:** gli endpoint di export non SHALL aprire connessioni in scrittura; le connessioni continuano a usare `mode=ro`.
- **Sicurezza:** nessuno degli endpoint nuovi accetta dati utente in scrittura; i parametri di filtro continuano a essere applicati con parametri posizionali.
- **Dipendenze nuove:** `Leaflet.markercluster` via CDN, `openpyxl` o `xlsxwriter` per l'export Excel lato Python (aggiungere a `web/requirements.txt`), `reportlab` per il PDF scheda ente (aggiungere a `web/requirements.txt`).
- **Carta intestata MS:** il PDF della scheda ente SHALL utilizzare lo stesso file `docs/PDF/MS_Carta_Intestata.pdf` già in uso per la documentazione tecnica, copiato o referenziato dentro `web/`.

Scenari (WHEN / THEN)

A. Mappa

Scenario A.1 — Mappa con tutti gli enti del filtro

- WHEN l'utente apre la vista mappa con `regione=Toscana`
- THEN la mappa mostra i marker di **tutti** gli enti toscani con coordinate, non solo i primi 20

Scenario A.2 — Clustering attivo

- WHEN due o più marker sono geograficamente vicini al livello di zoom corrente
- THEN i marker sono rappresentati da un cluster con il numero di enti contenuti; un click sul cluster ne effettua lo zoom in

Scenario A.3 — Filtro live da sidebar

- WHEN l'utente, nella vista mappa, deselecta la checkbox "Lombardia" dalla sidebar regioni

- THEN i marker lombardi spariscono immediatamente dalla mappa senza ricaricamento e la query string nell'URL viene aggiornata

Scenario A.4 — URL persistente

- WHEN l'utente condivide l'URL della mappa con filtri attivi a un collega
- THEN il collega aprendolo vede la stessa selezione di marker

B. Export

Scenario B.1 — Export CSV filtrato

- WHEN l'utente con filtro `sezione_registro="Associazioni di promozione sociale"` clicca "Esporta CSV"
- THEN scarica un file `.csv` UTF-8 con BOM e separatore `;` contenente solo gli APS

Scenario B.2 — Export Excel

- WHEN l'utente clicca "Esporta Excel"
- THEN scarica un `.xlsx` con un foglio "Enti", header congelato, larghezza colonne adeguata al contenuto

Scenario B.3 — Scheda PDF

- WHEN l'utente apre la scheda di CAI Pisa (`id_runts=83894`) e clicca "Scarica scheda PDF"
- THEN scarica un PDF con la carta intestata MS, intestazione "CLUB ALPINO ITALIANO SEZIONE DI PISA", tabella dei campi e mappa della sede

C. Scraper

Scenario C.1 — Rerun che preserva le coordinate

- WHEN lo scraper rilancia su un ente già geocodificato in precedenza
- THEN i valori di `lat` e `lon` nel DB restano invariati (non vengono azzerati)

Scenario C.2 — Recupero da timeout transitorio

- WHEN l'estrazione di un ente fallisce per timeout al primo tentativo, ma riesce al secondo
- THEN l'ente è correttamente salvato; il report finale conta 1 al "recuperato al 2° tentativo"

Scenario C.3 — Fallimento definitivo

- WHEN l'estrazione di un ente fallisce a tutti e 3 i tentativi

- THEN l'ente viene segnalato come errore nel log e nel report; lo scraper prosegue con il successivo senza interrompersi

D. Performance e cache

Scenario D.1 — Cache hit nel geocoder

- WHEN due enti hanno stessa terna (comune, provincia, regione) e il primo è già stato geocodificato in una run precedente
- THEN il secondo viene risolto da `geocoding_cache` senza chiamata HTTP a Nominatim; il report finale incrementa il contatore "da cache"

Scenario D.2 — Cache miss

- WHEN un ente ha una terna non presente in `geocoding_cache`
- THEN il geocoder interroga Nominatim, applica il risultato e popola la cache; il report incrementa "da Nominatim"

Scenario D.3 — Query filtrata veloce

- WHEN l'utente filtra la lista per `sede_regione="Lombardia"`
- THEN la query SQL usa `idx_enti_sede_regione` (verificabile con `EXPLAIN QUERY PLAN`)

Note di design

Endpoint GeoJSON vs JSON inline

Il template `list.html` oggi incorpora i dati come array JS inline. Per 226 enti questo è accettabile (~50 KB); con clustering e filtri client-side la stessa scelta resta valida. Tuttavia, separare l'endpoint `enti.geojson` permette riuso (es. embed esterni) e disaccoppia la mappa dal template di lista. **Scelta:** endpoint dedicato + chiamata fetch dal template.

Implementazione del MERGE su lat/lon (C1)

Due opzioni:

1. **Read-modify-write** in `upsert_ente` : prima di INSERT OR REPLACE, fare `SELECT lat, lon FROM enti WHERE id_runs = ?` e riempire il dict se mancanti.
2. `INSERT ... ON CONFLICT ... DO UPDATE SET ... WHERE excluded.lat IS NOT NULL` : usare la sintassi SQLite UPSERT con clausola condizionale.

La (2) è più atomica e SQL-nativa, ma richiede ripensare le 23 colonne dell'attuale `INSERT OR REPLACE`. La (1) è meno elegante ma più conservativa e facilmente testabile. **Scelta raccomandata:** (1) per la prima versione, eventualmente migrare a (2) in un rilascio successivo.

Cache key per geocoding (D2)

La chiave deve essere robusta a variazioni minori: comune in maiuscolo/minuscolo, spazi multipli, accenti. Implementazione proposta:

```
def cache_key(comune: str, provincia: str | None, regione: str | None) -> str:
    parts = [(comune or "").strip().lower(),
              (provincia or "").strip().lower(),
              (regione or "").strip().lower()]
    return "|".join(parts)
```

Sidebar mappa: stato in URL

Per la sincronizzazione URL dei filtri della sidebar (A4) usare i parametri esistenti `regione` e `sezione_registro` ma in formato multi-valore (comma-separated): es.

`?regione=Toscana,Lombardia&sezione_registro=APS` . Richiede minimo adattamento server-side per accettare anche valori multipli.

Excel: libreria

`openpyxl` è la scelta più semplice e maintained; `xlsxwriter` ha output più piccolo e supporta meglio formule. Per il nostro caso (singolo sheet, no formule), entrambe vanno bene. **Scelta:** `openpyxl` per coerenza con l'ecosistema Python più diffuso.

PDF scheda ente

Riutilizziamo il sistema già messo a punto per la documentazione tecnica: contenuto generato con `reportlab` (font DejaVu per Unicode), poi `pypdf.merge_page` con `MS_Carta_Intestata.pdf` come sfondo. Layout pensato per stare in una pagina singola; due pagine accettabili se i campi sono molti.

Alternative considerate

- **Leaflet senza clustering:** bocciato — illeggibile in città con 5+ sezioni CAI vicine.
- **Server-side filtering della mappa:** bocciato — la latency utente sarebbe sensibile, e il dataset è piccolo. Filtro client-side migliore.
- **WeasyPrint** invece di reportlab per il PDF della scheda: più semplice ma meno controllo su carta intestata; bocciato per coerenza con la pipeline doc esistente.

Rischi e mitigazioni

- **Compatibilità Leaflet.markercluster** con il setup attuale: testare con la versione 1.5.x stabile, fissarla nel template.
- **Dimensione GeoJSON** in crescita: con 226 enti il payload è ~30 KB; sopra 5000 enti valutare PBF/MVT.
- **Cache geocoding desallineata** se i confini comunali cambiano (raro ma possibile): aggiungere comando `python -m scraper.geocoder --refresh-cache` per invalidare.

- **PDF scheda ente con campi extra-lunghi** può sfiorare la pagina: implementare flowables con `KeepInFrame` di reportlab.

Allegati

Posizionare in `docs/releases/cairunts_001_assets/` (cartella da creare al primo allegato):

- `cairunts_001_assets/mockup_mappa_sidebar.png` — mockup del pannello laterale dei filtri sulla mappa.
- `cairunts_001_assets/mockup_export_buttons.png` — mockup della toolbar export nella lista.
- `cairunts_001_assets/mockup_scheda_pdf.png` — anteprima della scheda ente PDF.
- `cairunts_001_assets/diagramma_retry_scraper.png` — flowchart del retry con backoff.

Tasks tecniche

A. Mappa potenziata

- [] A.1.1 Aggiungere route `GET /api/enti.geojson` in `web/app.py` che riusa la logica filtri di `enti_list`.
- [] A.1.2 Spostare la costruzione dell'array marker dal template a una funzione `fetch_geojson()` nel JS della pagina lista.
- [] A.2.1 Includere `leaflet.markercluster` JS+CSS via CDN nel template `list.html`.
- [] A.2.2 Sostituire `L.marker(...).addTo(map)` con `L.markerClusterGroup()` nel JS della mappa.
- [] A.3.1 Aggiungere markup HTML della sidebar in `list.html` (collassabile su mobile).
- [] A.3.2 Aggiungere conteggio dinamico per ogni voce (calcolato lato JS dal geojson).
- [] A.3.3 Implementare il filtro client-side dei marker al cambio checkbox.
- [] A.4.1 Sincronizzare la query string del browser con `history.replaceState` al cambio filtri.
- [] A.5.1 Estendere `web/app.py` per accettare valori multipli comma-separated nei parametri `regione` e `sezione_registro`.

B. Export

- [] B.1.1 Implementare `GET /api/enti.csv` con `StreamingResponse` per CSV UTF-8 BOM + `;`.
- [] B.2.1 Aggiungere `openpyxl` a `web/requirements.txt`.

- [] B.2.2 Implementare `GET /api/enti.xlsx` con generazione in memoria via `openpyxl.Workbook` + `BytesIO` .
- [] B.3.1 Estrarre la logica di filtro in funzione condivisa con `enti_list` .
- [] B.4.1 Aggiungere `reportlab` (+ `pypdf`) a `web/requirements.txt` .
- [] B.4.2 Implementare `GET /ente/{id_runts}/pdf` riutilizzando lo script `build_pdf.py` adattato a singolo ente.
- [] B.4.3 Copiare `MS_Carta_Intestata.pdf` dentro `web/static/` (o caricarlo da path configurabile).
- [] B.5.1 Aggiungere i tre pulsanti nei template Jinja con icone Bootstrap.

C. Scraper resilienza

- [] C.1.1 In `scraper/db.py` , modificare `upsert_ente` per leggere `lat` , `lon` esistenti prima dell' `INSERT OR REPLACE` se non presenti nel dict.
- [] C.1.2 Aggiungere test in `scraper/test_upsert.py` che verifica preservazione lat/lon.
- [] C.2.1 In `scraper/scraper.py` , avvolgere il blocco di estrazione di un singolo ente in un loop `for attempt in range(3)` con `await asyncio.sleep(2 ** attempt)` su eccezione.
- [] C.3.1 Mantenere il comportamento attuale di fallback (log error + continue) dopo il 3° tentativo.
- [] C.4.1 Aggiungere contatori `recovered_attempt_2` , `recovered_attempt_3` , `failed_after_retry` al report finale di `main.py` .

D. Performance e cache

- [] D.1.1 In `scraper/db.py` , aggiungere `CREATE INDEX IF NOT EXISTS idx_enti_sede_regioni ON enti(sede_regioni)` e analogo per `sezione_registro` allo schema, oltre a inserirli in `_MIGRATIONS` .
- [] D.2.1 Aggiungere la `CREATE TABLE IF NOT EXISTS geocoding_cache (...)` allo schema in `scraper/db.py` .
- [] D.3.1 In `scraper/geocoder.py` , implementare `cache_key(comune, provincia, regione)` e `lookup_cache(conn, key)` .
- [] D.3.2 Prima di `_nominatim_query` , eseguire `lookup_cache` ; se hit, usare le coordinate.
- [] D.4.1 In caso di successo Nominatim, eseguire `INSERT OR REPLACE INTO geocoding_cache (...)` .
- [] D.5.1 Estendere il report finale del geocoder con contatori `from_cache` e `from_nominatim` .

Verifica

- [] V.1 Test manuale: rilanciare scraper su 3-4 enti già geocodificati e verificare che `lat / lon` siano ancora presenti.
- [] V.2 Test manuale: aprire la mappa con filtro regionale e verificare che tutti i marker della regione siano visibili.
- [] V.3 Test manuale: esportare CSV e Excel, aprirli in LibreOffice/Excel e verificare encoding + caratteri italiani.
- [] V.4 Test manuale: generare scheda PDF di CAI Pisa, verificare layout + mappa + carta intestata.
- [] V.5 `EXPLAIN QUERY PLAN SELECT ... WHERE sede_regione = 'Toscana'` deve mostrare uso di `idx_enti_sede_regione`.
- [] V.6 Eseguire geocoder due volte di fila: il secondo run deve avere `from_cache` > 0 e `from_nominatim` ridotti rispetto al primo.

Note per OpenSpec

Il `change` OpenSpec corrispondente potrebbe essere `cairunts-001-mappa-export-resilienza` con quattro delta-spec separati (uno per macro-tema A/B/C/D), ciascuno con i propri Requirement / Scenario nel formato canonico `### Requirement: ... + #### Scenario: ...`. La maggior parte dei requisiti qui sopra è già nel formato compatibile.